Missing Production Migrations

Project: TSCopier **Environment:** Production `sxkpcovbyaficvtkpsdo` **Date:** 2026-08-16
Verification: direct DB checks, staging control group

Summary

Production is **behind staging by 4 migrations**. Two are a **live bug** — deployed code writes to a table that does not exist on prod. The other two are groundwork for a **dormant** (kill-switched) feature and are safe to defer.

#	MIGRATION	ON PROD	PRIORITY	REASON
1	`20260810000000_trade_reports`	MISSING	HIGH	Base table for trade reports — deployed code writes to it
2	`20260816000000_trade_reports_signal_id`	MISSING	HIGH	Depends on #1; adds `signal_id` column
3	`20260730120000_layering_modes_foundation`	MISSING	MEDIUM	Additive foundation columns (feature dormant)
4	`20260731120000_layering_plans`	MISSING	MEDIUM	Layering blueprint feature (dormant); depends on #3

> **Already on prod:** `enforce_plan_broker_channel_limits` (registered as `20260805082647`); functions and triggers verified live since 2026-08-05.

What each missing migration does

1. trade_reports HIGH — live bug

Creates the `trade_reports` table — the queue where users file complaints about trades ("wrong entry", "wrong SL/TP", "not executed", etc.) for support staff to review. Both reporting paths insert into this table:

- The **manual "Report" button** in the trade detail modal (`src/components/trades/ReportTradeModal.tsx` — client-side INSERT).
- The **in-app assistant** `report_trade` tool (`supabase/functions/assistant-chat/index.ts`).

What it creates:

- `trade_reports` table: `user_id`, trade snapshot (`symbol`, `direction`, `ticket`, `broker_label`, `entry_price`, `sl`, `tp`, `lot_size`), `category`, `reason`, `status` (open/resolved), timestamps.
- Row Level Security: users can insert/view only their own reports.
- Indexes on `(user_id, created_at)` and `(status)`.

> **Impact without it:** any attempt to file a trade report fails with `ERROR 42P01: relation "trade_reports" does not exist`. Live, user-facing defect on prod.

2. trade_reports_signal_id HIGH

Adds a `signal_id` column so reports stay traceable back to the originating signal — including reports on **skipped / non-actionable** trades that have no symbol or ticket (which the assistant can now file).

What it creates:

- `trade_reports.signal_id` (nullable, backwards compatible).
- Index `trade_reports_signal_idx` on `signal_id`.

Impact without it: once #1 is applied, reports still work — they just can't link to a signal. Required for the assistant's skipped-trade reporting fix.

3. layering_modes_foundation MEDIUM — dormant

Prepares the `range_pending_legs` table (the ladder of pending orders for range trades) for future "static/dynamic" layering modes, where the ladder is planned up-front instead of built the legacy way. Only **adds empty columns** — existing rows are untouched and keep behaving exactly as before.

What it creates:

- `range_pending_legs.layer_plan_id` (text, nullable).
- `range_pending_legs.layer_plan_metadata` (jsonb, nullable).
- Partial index on `layer_plan_id`.

4. layering_plans MEDIUM — dormant

Creates the `layering_plans` table — the "blueprint" for static/dynamic range-layering ladders (mode, status lifecycle, frozen calculator metadata, semantic fingerprint), plus the `activate_layering_plan` function that materializes plans into executable legs. Adds many **nullable** columns to `range_pending_legs` and unique indexes preventing duplicate ladder steps.

What it creates:

- `layering_plans` table (service-role only; no client access).
- `activate_layering_plan(...)` function + layering-settings guard triggers.
- Unique indexes: `(signal_id, broker_account_id, basket_key, mode)`, `(layer_plan_id, step_idx)`, `(broker_account_id, broker_client_reference)`.

- 17 additive nullable columns on `range_pending_legs` (`broker_client_reference`, `native_submission_status`, `submission_*`, `cancellation_*`, `reconciliation_*`, …).

Dependency: requires #3 first — its unique index on `range_pending_legs(layer_plan_id, step_idx)` needs the `layer_plan_id` column. Feature is behind kill-switch flags (`LAYERING_MODES_EXECUTION_ENABLED`, `LAYERING_MODES_PREPARE_ONLY`), restricted to staging allowlisted accounts.

Relevance to the codebase (plain English)

What "the bot" is

By "the bot" this document means the **worker** — the software that runs the whole copy-trading machine on a server called Railway. There are two parts:

- **The Listener** — sits inside Telegram and watches your signal channels. When a channel posts "Buy XAUUSD now", the listener grabs it and decides if it is a real trade signal or just a promo message.
- **The Trade Worker** — takes a confirmed signal and sends the actual order to your broker (through the FxSocket bridge into MT4/MT5). It handles entries, take-profits, stop-losses, baskets (multi-leg trades), and management messages.

In one line: the worker is the robot that turns a Telegram message into a real trade on your trading account. The Trade Worker is the one that places range-layering orders.

Why these migrations matter for the worker

These two migrations are not new ideas — they are the "database paperwork" for a trading feature the worker already knows how to do. Think of it like a contractor building a house:

- **Migration 3** gives every waiting order a place to write down "I belong to blueprint #123" — so the worker can tell which orders came from which plan.
- **Migration 4** creates the actual blueprint *file* (the plan: how many orders, at what prices) and gives each waiting order extra blank boxes to fill in as it moves through the process (submitted, waiting, confirmed, cancelled).

Why the worker breaks without them

The worker's code is already written to use these boxes. Today:

- When a plan is activated, the worker writes the plan to a database table that **does not exist on prod** → the write fails.
- When it submits a pending order, it records the status in a **column that does not exist on prod** → the write fails.
- If the worker restarts mid-trade, it reads the saved plan to continue — but there is **nowhere to read it from** → it cannot recover.

> **Bottom line:** the migrations are the shelves and files the already-written software expects to find. **Staging has them; prod does not.** As long as the feature is switched off, nothing breaks.

> The moment anyone turns layering on, prod breaks — because the software would try to use space that was never created.

Verification evidence

Same 5 queries run against **staging** (control — applied) and **prod**:

CHECK	STAGING (CONTROL)	PROD
`to_regclass('public.trade_reports')`	trade_reports	null
`information_schema.tables (%report%)`	found	[]
`pg_tables (%trade_report%)`	found	[]
`schema_migrations (trade_reports)`	both versions	[]
`SELECT count(*) FROM trade_reports`	works (8 rows)	42P01 error

Additional prod checks: `layering_plans` → `to_regclass` returns `null`; `range_pending_legs` has no `layer_plan_id` / `layer_plan_metadata` / `broker_client_reference` columns; no layering guard triggers present. The staging control proves the query method is sound — the prod negatives are real.

What has to be done

Recommended: apply #1 and #2 to prod now — they fix the live trade-report failure for both the manual modal and the assistant.

1. Apply SQL via the Supabase Management API: `POST /v1/projects/{ref}/database/query` with `Authorization: Bearer <token>` and body `{"query": "<sql>"}`.
2. Register in `supabase_migrations.schema_migrations`: `INSERT ... VALUES ('{version}', ARRAY['-- {name}'], '{name}') ON CONFLICT (version) DO NOTHING`.
3. Verify objects exist (table, column, indexes) — mirror the evidence above.

Decision needed: whether to also apply #3 and #4 to prod. They are additive and dormant; apply to keep prod in lockstep with staging, or defer until the layering feature is scheduled for rollout.

Also pending (code, not migration): redeploy the `assistant-chat` edge function to staging then prod with the skipped-trade `report_trade` fix (committed in the working tree; the function is deployed manually).

Plain English: TSCopier's production database is missing 4 things. Two of them (the trade-reports table) mean users can't file a trade complaint right now — the button errors out. The other two are behind-the-scenes prep for a future feature that isn't turned on yet. Staging has all 4.